

第十二章：论文串读——GPT 系列演进与 LLaMA / Qwen 的现代改动

上一章我们逐节读完了 2017 年那篇奠基论文，末尾留了一张「2017 → 今天」的对照表：整体架构从两栈变成 decoder-only，归一化从 Post-LN 换到 Pre-LN、从 LayerNorm 换到 RMSNorm，FFN 的激活从 ReLU 换成 SwiGLU，位置编码从 sinusoidal 换成 RoPE，注意力从 MHA 换成 GQA。那张表的右列，就是这一章要逐项展开的全部内容。

这一章咱们不再读单独一篇论文，而是顺着时间线串读一批论文——不抠某一篇的每个公式，而是把同一条技术脉络上的几篇关键论文摆在一起，看清每一代相对上一代到底改了什么、为什么改。我们串两条线：

- 第一条线（第 2-5 节）：GPT-1 → GPT-2 → GPT-3，看 decoder-only 这条路线是怎么一步步被确立的——从「预训练 + 微调」到「零样本」，再到「few-shot 上下文学习」，规模一路放大到质变。
- 第二条线（第 6 节）：从原版 Transformer 到 LLaMA / Qwen，把现代大模型相对原版换掉的那批零件（RoPE / RMSNorm / SwiGLU / GQA / MoE）逐项拆开对照。

需要提前说明：这两条线上用到的零件，前面各章几乎都已经介绍过了——RoPE 在第 4 章第 6 节、GQA 在第 7 章第 5 节、RMSNorm / SwiGLU 在第 8 章第 4-5 节、decoder-only / Pre-LN / MoE 在第 9 章、in-context learning 也在第 9 章第 4 节提到过。所以本章不做重复推导，侧重于那条贯穿多篇论文的演进叙事，以及「哪一项改动是哪一篇论文、哪一年引入的」这张时间地图。

实战依旧 全程 CPU：不训练、不推理，只用 `AutoConfig` 读几个真实模型的 config.json（GPT-2、Qwen3-8B），把「论文里说的改动」和「config 里的字段」一一对上，最后画一张 GPT-1/2/3 参数量增长图。

想直接跑示例？点这里  [Open in Colab](#)。

硬件门槛：概念章，CPU 即可 ✓。本章实战只用 `AutoConfig` 下载几 KB 的 config.json 并画一张小图，Colab 免费 CPU 运行时秒级完成，不需要 GPU、也不下载任何模型权重。

一、承上启下：把上一章那张对照表展开

1.1 两条主线：先立路线，再换零件

第 11 章我们把 2017 原版 Transformer 从头读了一遍，也指出了它和今天大模型的差距。可从「2017 的原版」到「今天的 Qwen3」，中间不是一步跨过去的——是一连串论文，每篇改一两件事，攒了七八年才成今天这个样子。

这条路可以清清楚楚地分成两段，正好对应本章的两条线：

- 先立路线：原版是为翻译设计的 encoder-decoder 两栈。OpenAI 的 GPT 系列（2018-2020）做的事，是把它裁成 decoder-only 一栈，并证明「这一栈 + 大规模预训练」这条路能到达什么高度。这一段的关键词是架构选型和规模，不怎么动零件本身。
- 再换零件：decoder-only 这个骨架定下来之后，2020 年往后的改进就不再动「一个栈还是两个栈」了，而是在这个固定骨架上，把内部的零件一个个换成更好的版本——位置编码换 RoPE、归一化换 RMSNorm、FFN 换 SwiGLU、注意力换 GQA、FFN 有时还换成 MoE。LLaMA（Meta，2023）和 Qwen（阿里，2023 起）是这一段最有代表性的开源模型。

一句话概括：GPT 系列定下了「用哪种骨架、做到多大」，LLaMA / Qwen 定下了「这个骨架里每个零件用什么型号」。下面这张图先把全章的路线摆出来——上半部分是两条线上的六个节点、每个节点相对上一代的关键增量改动（delta），下半部分是那条八年未变的 decoder-only 骨架：



1.2 怎么读一串「系列论文」

读单篇论文（第 11 章）和读一串系列论文，方法不太一样。读单篇是「把这一篇的每个设计搞懂」；读系列则是盯住每一代相对上一代的增量改动（delta）——每一代论文你只需要问三个问题：

1. 上一代的瓶颈是什么？（这一代想解决什么）
2. 这一代改了哪几件事？（架构、数据、规模，通常就一两处关键改动）
3. 改完之后多出了什么新能力？（zero-shot、few-shot、更长上下文……）

抓住这三问，你不用逐字读完每篇论文，也能把一条技术线的来龙去脉讲清楚。下面读 GPT-1/2/3 就用这个框架：每一节开头先说它相对上一代的 delta，再展开细节。

二、GPT-1 (2018) : decoder-only 与「预训练→微调」范式

这一代的 delta: 第一次证明「在海量无标注文本上用 Transformer decoder 做自回归预训练, 再在下游任务上微调」这套两阶段范式能大幅超过从零训练的专用模型。它奠定了后面所有 GPT 的骨架——decoder-only。

GPT-1 的论文题目是《Improving Language Understanding by Generative Pre-Training》(Radford 等, OpenAI, 2018)。“Generative Pre-Training”缩写就是 GPT。

2.1 论文要解决的问题: 标注数据太贵

2018 年之前, NLP 做一个具体任务 (情感分类、蕴含判断、问答……) 的通常做法是: 先为这个任务专门人工标注一批数据, 再拿这批数据从零训一个模型。问题是带标签的数据又少又贵 (每一条都要人工标), 而无标签的文本几乎无限 (整个互联网)。论文的核心动机就一句话: 能不能先在海量无标签文本上把语言的通用规律学出来, 再用少量标注数据微调到具体任务上?

这正是第 10 章讲的自回归语言建模目标能派上用场的地方——预测下一个 token 这件事不需要任何人工标注, 语料自己就是答案 (第 t 个 token 的「标签」就是语料里第 $t+1$ 个 token)。所以拿它做预训练, 等于免费从无标注文本里获取监督信号。

2.2 两阶段范式: 无监督预训练 + 有监督微调

GPT-1 的方法就两个阶段, 这套范式一直影响到今天:

- 阶段一 (预训练): 在 BooksCorpus (约 7000 本书、 ≈ 5 GB 文本) 上做标准的自回归语言建模——最大化 $\sum_t \log P_\theta(w_t | w_{t-k}, \dots, w_{t-1})$, 也就是第 10 章那个「让真实语料最可能」的目标。模型是一个 12 层的 Transformer decoder。
 - 这个式子照抄自论文, 有两处值得说明。一是求和范围: 它是从语料第一个 token 起, 每个位置都算一遍。二是条件里为什么不是全部前文: 第 10 章写的是 $P_\theta(w_t | w_{<t})$ ($w_{<t}$ 指 t 之前的全部前文), 论文这里写成 $w_{t-k}, \dots, w_{t-1}$, 其中 k 是上下文窗口大小 (GPT-1 取 512)。原因是: BooksCorpus 是几千本书首尾相连拼成的一整段超长文本, 而模型的上下文长度是有限的, 不可能在预测第一百万个词时还回看到语料的第一个词, 所以条件被截断成「最近的 k 个 token」。开头那几个位置前文不足 k 个, 就有多少用多少。换句话说, 两个式子是同一个目标, 只是第 10 章写的是理想形式、论文写的是有限上下文下的实际形式。
- 阶段二 (微调): 把预训练好的模型接一个小小的任务头 (比如分类就接一个线性层), 在下游任务的标注数据上继续训。论文的一个技巧是把不同任务的输入都改写成一段连续的 token 序列 (用特殊分隔符拼接), 这样同一个 decoder 骨架不用改结构就能适配多种任务——这已经有点「一切皆序列」的味道了。
 - 「接一个任务头」具体怎么接: 把序列喂进预训练模型, 取最后一个 token 在最后一层的隐藏状态 h (形状 `[d_model]`, GPT-1 是 768), 再过一个新加的线性层 W_y (形状 `[d_model, C]`, C 是这个任务的类别数), softmax 一下就得到类别分布: $P(y | x) = \text{softmax}(hW_y)$ 。整个微调阶段新增的

参数只有 W_y 这一个矩阵（外加下面那几个分隔符的 embedding），骨架里原有的权重全部沿用预训练结果，并和 W_y 一起更新。论文还发现一个实用技巧：微调时把预训练用的语言建模目标当作辅助损失一起优化（总损失 = 任务损失 + $\lambda \times$ 语言建模损失），泛化更好、收敛也更快。

- 输入怎么「改写成一条序列」：论文用三个特殊 token——起始 `<s>`、分隔符（论文里用 `$`，下面记作 `<sep>`）、结尾 `<e>`——把结构化输入拍平，换任务只需要换拼接方式和那个线性层，decoder 骨架一字不动：

- 分类： `<s>` 文本 `<e>`
- 蕴含判断（判断前提能否推出假设）： `<s>` 前提 `<sep>` 假设 `<e>`
- 相似度（判断两句话是否同义）：两句按两种先后顺序各拼一条、分别过模型，得到的两个 h 相加后再进线性层——因为「相似」本身没有先后之分，这样拼保证换个顺序结果不变
- 多选题：每个候选答案各拼一条 `<s>` 上下文 `<sep>` 候选答案 `<e>`，各自算出一个分数，最后在所有候选之间做一次 softmax 选出答案

结果是：在当时的 12 个 NLP 任务里，有 9 个刷新了最好成绩。「预训练一个通用模型 + 轻量微调到下游」这套路子，从此成了 NLP 的主流范式——今天的 SFT（第 28-29 章）/ LoRA（第 37-38 章）本质上还是这套两阶段思路的延续，只不过预训练模型换成了几百上千亿参数的大模型。不过要提醒一句：延续的是「先预训练、再用少量标注数据适配」这个范式，具体做法早已不同——GPT-1 是每个下游任务单独接一个任务头、单独训一份专属模型（12 个任务就是 12 份权重，输出是类别标签）；今天的 SFT 不接任何任务头，输出仍然是下一个 token，训的是「跟着指令答话」这一种通用行为，一份模型通吃所有任务，LoRA 之类更是连主干权重都不动、只训旁挂的低秩矩阵。

2.3 为什么砍掉 encoder，只留 decoder

原版 Transformer 是 encoder + decoder 两栈（第 9 章第 3 节），GPT-1 只取了 decoder 那一栈（去掉 cross-attention 后的版本）。为什么这么裁？第 9 章第 4 节已经把缘由讲清楚了，这里简单回顾下结论：

- 训练信号最密：因果语言建模让一段长度 L 的文本里，几乎每个 token 都是一次「预测下一个」的监督，一次前向产生 $L - 1$ 条监督信号，数据利用率拉满。
- 推理最省：因果掩码保证「前面 token 的表示不依赖后面」，天生适配 KV cache（第 14 章）。
- 任务最统一：一切都能写成「给一段前文、续写下去」，不用像 encoder-decoder 那样为每类任务设计输入输出格式。

GPT-1 时还看不出这三点有多决定性（那会儿它只是「效果不错的一种选择」），但正是这三点，让 decoder-only 在后面几年最终成为事实标准。

2.4 顺带一提：同期的 BERT 走了另一条路

几乎同时（2018 年底），Google 的 BERT 用的是 encoder-only 栈 + 掩码语言建模（MLM）——随机盖住句子里一部分 token、让模型同时看左右两边去猜被盖住的词。BERT 是双向的（每个位置能看到整句话），在「理解类」任务（分类、抽取、检索）上一度全面压过 GPT。

所以 2018 那会儿其实是两条路线并行：GPT 走「单向 + 生成」，BERT 走「双向 + 理解」。为什么最后是 GPT 这条单向路线通向了今天的大模型？核心就是第 9 章第 4 节讲的：单向的因果语言建模能把所有任务统一成「续写」，且规模放大到足够大以后 in-context learning 会自己涌现——而这两点恰恰是 GPT-2、GPT-3 接下来要证明的事。BERT 那条路今天仍出现在检索 / embedding 模型里，但通用大模型的主赛道被 GPT 这条路拿下了。

三、GPT-2（2019）：规模变大，零样本登场

这一代的 delta：骨架几乎不变，把规模和数据大幅拉大（最大 1.5B 参数、40 GB 的 WebText），并提出一个大胆主张——足够强的语言模型不微调也能直接做任务（zero-shot，零样本）。架构上只做了三处工程性的小改动，其中最重要的一处是把 Pre-LN 定为后续模型的默认配置。

GPT-2 的论文题目是《Language Models are Unsupervised Multitask Learners》（Radford 等，OpenAI，2019）。

3.1 核心主张：语言模型是无监督的多任务学习器

GPT-1 还需要「阶段二微调」才能做下游任务。GPT-2 想更进一步：如果预训练语料足够大、足够杂，模型是不是根本不用微调，直接就会做很多任务？

背后的直觉是：互联网文本里本来就天然包含各种任务的示范。翻译任务的语料里会自然出现「英文句子（法文翻译）」这样的配对；问答的语料里会出现「问题？答案。」这样的模式。所以一个在足够杂的语料上训练的语言模型，其实是在顺带地学会了一大堆任务——你只要在 prompt 里把任务描述清楚（比如给一段英文、结尾写 "translation to French:"），它就能续写出答案。这就是标题说的「无监督的多任务学习器」。

GPT-2 用这套思路做了 zero-shot（零样本）测试：不给任何微调、不给任何示例，仅靠一句任务提示，就去做翻译、问答、摘要。效果谈不上惊艳，但证明了「规模够大时，任务能力会从纯语言建模里自己冒出来」——这颗种子，到 GPT-3 就长成了参天大树。

3.2 架构上的三个小改动（Pre-LN 就在这里定型）

GPT-2 的模型结构和 GPT-1 基本一样，论文里只提了三处工程性调整。前两处第 8、9 章都讲过原理，第三处是本章第一次出现、下面顺带说清——它们都是从 GPT-2 这一代起，成为后来大模型的默认配置的：

- LayerNorm 挪到子层输入端（Pre-LN）：原版和 GPT-1 是 Post-LN（ $\text{LayerNorm}(x + \text{Sublayer}(x))$ ），GPT-2 把 LayerNorm 移到每个子层之前（ $x + \text{Sublayer}(\text{LayerNorm}(x))$ ）。原理见第 8 章第 3 节（残差高速公路）与第 9 章第 5 节（Pre-LN vs Post-LN）：Pre-LN 让残差成为一条干净的恒等高速公路，深层才训得稳。
- 最后一个 block 之后再加一道 LayerNorm（final norm）：Pre-LN 的残差流会一路累加、幅度越来越大，所以在输出头（代码里叫 `lm_head`）之前补一道归一化把它拉回正常尺度。第 9 章第 2 节画的那条流水线里，**Final Norm** 这一环就是它。

- 残差投影的初始化按 $1/\sqrt{N}$ 缩放（ N 为残差层数，约 $2 \times$ block 层数——每个 block 有注意力、FFN 两条残差）：具体做法是把每条残差支路末端那个输出投影矩阵（注意力的 W_O 、FFN 的 W_{down} ，现代实现里叫 `o_proj` / `down_proj`，GPT-2 原始实现里两处都叫 `c_proj`）的初始权重乘上 $1/\sqrt{N}$ ，再开始训练。这是个纯粹的训练稳定性技巧。
- 为什么要乘这一下：Pre-LN 下子层「读流」时先归一化，但写回流的增量 Δ 是未归一化的（第 9 章第 5.3 节），而这些增量彼此近似独立，方差逐条相加—— N 条累加下来，残差流的方差就涨到约 N 倍，深层激活值的幅度随之失控膨胀。把这些投影矩阵的初值按 $\sqrt{N}$ 缩小，它们算出的增量 Δ 也随之变小，每条的方差变成原来的 $1/N$ ， N 条加起来总方差正好回到 $O(1)$ ，跟单层时同量级。
- 它和上一条 final norm 分工不同、互相替代不了：final norm 只在整个栈的末端动一次手，管的是交给输出头的那个向量尺度；初始化缩放作用在全部 N 条残差支路上，管的是沿途每一层的幅度。而且 final norm 是等比例缩放，改不掉「越靠后的层，其增量占残差流的比例越小」这个层间失衡。时机上两者也差一截：final norm 是带可学习参数的常驻组件，训练和推理时一直在起作用；而 $1/\sqrt{N}$ 只在训练开始前动一次手，缩小的是这些投影矩阵的初值——训练跑起来后权重随梯度更新、初值不再保留，但它撑住的那段稳定状态正好覆盖开头最容易发散的几百步。一个是常驻部件，一个是起跑姿势。

除此之外，GPT-2 还把上下文长度从 512 提到 1024、词表提到 50257、用 byte-level BPE（第 3 章的 BBPE）。注意这里在模型结构上没有任何一处是「发明新组件」——全是把已有零件调参、换位置、扩规模（真要说新东西，是 tokenizer 那一侧的 byte-level BPE，而它不属于 block 内部的结构）。

四、GPT-3（2020）：few-shot 上下文学习与规模的质变

这一代的 delta：把规模再放大两个数量级到 175B，骨架依旧几乎不动，却涌现出一个谁也没直接训练过的新能力——in-context learning（上下文学习）：在 prompt 里给几个示范（few-shot），模型不更新任何参数就能照着做新任务。这是「量变引起质变」最经典的一个案例。

GPT-3 的论文题目是《Language Models are Few-Shot Learners》（Brown 等，OpenAI，2020）。光看标题就知道，它接着 GPT-2「zero-shot」的话头，往前推到了「few-shot」。

4.1 175B：把规模再放大两个数量级

GPT-3 最大的型号有 1750 亿（175B）参数、96 层、 $d_{\text{model}} = 12288$ 、96 个注意力头，在约 3000 亿 token 的语料上训练。相比 GPT-2 的 1.5B，参数量涨了 100 多倍。架构上唯一值得一提的改动是注意力层交替使用稠密（dense）和局部带状稀疏（locally banded sparse）注意力——所谓「带状」，是指每个 token 只关注邻近一段固定宽度的位置，注意力矩阵上只保留对角线附近的一条带，算力和显存都随之下降；GPT-3 让稀疏层和稠密层逐层交替，用稠密层补回全局视野。这个做法出自 Sparse Transformer（Child 等，2019），思路和第 23 章要讲的 sliding window 一脉相承，不过 LLaMA / Qwen 这条主线并没有沿用它——今天的 LLaMA / Qwen 用的都是稠密注意力，省显存这件事改由 FlashAttention（第 16 章）和 GQA 接手。除此之外，还是那套 decoder-only + Pre-LN 骨架。

这里第一次清晰地显现出一件事：这套骨架的主要「旋钮」就是规模——把宽度（ d_{model} ）、深度（层数）、头数一起往上推，模型能力就随之增长。至于「推多大最划算」，是第 21 章 scaling law 要回答的问题；GPT-3 用行动给出的答案是「再大两个数量级，会有惊喜」。

4.2 in-context learning：不更新参数就能学新任务

GPT-3 论文最重要的发现，是 in-context learning（上下文学习，也叫上下文内学习）。

传统的「学一个新任务」= 拿这个任务的数据去更新模型参数（训练 / 微调）。in-context learning 完全不动参数，而是把示范直接写进 prompt，让模型在这一次前向推理里「现学现用」。按 prompt 里给几个示范，分成三档：

- zero-shot（零样本）：只给任务描述，不给示例。如 把下面这句翻成法语：I love you =>
- one-shot（单样本）：给 1 个示例再让它做。
- few-shot（少样本）：给几个（通常几到几十个）示例再让它做。如：

把英语翻成法语：
sea otter => loutre de mer
cheese => fromage
I love you => ← 模型在这里续写出 "je t'aime"

关键在于：从头到尾没有任何一次反向传播、没有更新一个参数。模型只是「读到」prompt 里的几个示范，就在这一次生成里照着模式续写。这几乎重写了 NLP 的使用方式——过去用一个模型要先为每个任务训练，现在只要会写 prompt。后来的 prompt 工程、few-shot、CoT（chain-of-thought，思维链：让模型把推理步骤一步步显式写出来再给答案）全都建立在这个能力之上。

为什么 in-context learning 会出现？至今没有完全定论，但有一个朴素的直觉：第 3.1 节说过，海量语料里天然混着各种任务的示范模式；模型大到一定程度，就学会了识别 prompt 里正在演示的是什么模式、并把它延续下去这种更抽象的能力。而这个能力随规模增大才逐渐明显——小模型 few-shot 几乎没有增益，大模型才有。这正是「规模带来质变」的典型证据，也是 GPT-3 之所以震动整个领域的原因。

4.3 GPT 三代关键指标对照表

指标	GPT-1（2018）	GPT-2（2019，最大型号）	GPT-3（2020，最大型号）
参数量	117 M	1.5 B	175 B
层数	12	48	96
d_{model}	768	1600	12288

指标	GPT-1 (2018)	GPT-2 (2019, 最大型号)	GPT-3 (2020, 最大型号)
注意力头数	12	25	96
上下文长度	512	1024	2048
训练数据	BooksCorpus (≈ 5 GB)	WebText (≈ 40 GB)	≈ 570 GB (≈ 300 B token)
归一化位置	Post-LN	Pre-LN	Pre-LN
位置编码	learned 绝对	learned 绝对	learned 绝对
代表能力	预训练 + 微调	zero-shot	few-shot (in-context learning)

从 GPT 的三代模型可以看到：一是参数量三代涨了约 1500 倍，而架构上的实质改动只有 GPT-2 那处 Post-LN $\rightarrow$ Pre-LN（GPT-3 那处交替稀疏注意力没被后续继承，不计在内）——规模是主旋律、架构基本冻结；二是位置编码三代都是 learned 绝对位置编码（第 4 章第 5 节），还没换成 RoPE——RoPE 是下一条线（LLaMA / Qwen）才引入的。

五、小结：decoder-only 路线是怎么确立的

把 GPT-1/2/3 这条线用一句话收束：

GPT-1 证明了「decoder-only + 预训练」这条路能走通，GPT-2 证明了「规模够大能不微调直接做任务」，GPT-3 证明了「规模再大两个数量级会涌现 in-context learning」。三代下来，decoder-only 从「一种不错的选择」变成了「通用大模型的事实标准」。

到 GPT-3 之后，「一个栈还是两个栈」这个问题基本没人再争了——decoder-only 赢下了通用大模型这条主赛道（第 9 章第 4 节详细论证过）。

读到这里你多半想问一句：那从 GPT-3 到 ChatGPT 的那一步呢？为什么这条线到 GPT-3 就停了？因为那一步不在架构上。GPT-3 虽然会 few-shot，但它骨子里只会「续写」——你丢给它一个问题，它可能顺手再给你编三道类似的题目，而不是回答。把它变成一个听得懂指令、答得像助手的模型，靠的是 InstructGPT (2022) 那条后训练路线：先用人写的示范数据做 SFT（有监督微调），再用人类偏好训一个奖励模型、拿它去做 RLHF（基于人类反馈的强化学习）。模型骨架一个零件都没动，动的是训练流程。ChatGPT 就是这套后训练配方的产物，而后训练与对齐本身就是完整的一个阶段，留到第 27-36 章专门讲。

接下来几年，社区的注意力从「选哪种架构」转向了「在这个已经定型的骨架里，把每个零件换成更好的版本」。这就是第二条线——LLaMA / Qwen 的现代改动。

六、LLaMA / Qwen 的现代改动逐项拆解

2023 年 Meta 开源 LLaMA、阿里开源 Qwen，把「decoder-only 骨架 + 一套现代零件」的配方推广到了整个开源社区。这套配方相对原版 Transformer（乃至 GPT-3）换掉的零件，前面各章几乎都介绍过，所以这一节每个零件只做一个简单回顾，重点是把它们凑成一张完整的对照表。

先看它们共同的骨架长什么样——这也是今天绝大多数开源大模型的标准配置：

```
input_ids
  → Token Embedding
  → N × Decoder Block:
      x' = x + GQA(RMSNorm(x), 带 RoPE)
      out = x' + SwiGLU_FFN(RMSNorm(x')) ← MoE 版把这个 FFN 换成「路由器 + 多个专家」
  → Final RMSNorm
  → lm_head → logits
```

对照第 9 章第 2 节那条 decoder-only 流水线，骨架一模一样（embedding → N 个形状守恒的 block → final norm → lm_head），变的全是 block 内部每个零件的型号。下面逐个说。

6.1 RoPE：绝对位置编码 → 旋转式相对位置

一句话回顾（详见第 4 章第 6 节）：RoPE（旋转位置编码）不在输入上加一个位置向量，而是在每一层的 Q、K 上按位置施加一个旋转，让两个 token 的注意力分数只依赖它们的相对距离。

换在哪：原版和 GPT-1/2/3 用的都是绝对位置编码（sinusoidal 或 learned，第 4 章第 4-5 节）——位置信息在输入端一次性加好。LLaMA / Qwen 换成 RoPE，好处是相对位置天然更贴合语言（「第 5 个词和第 3 个词隔了 2 位」比「它俩分别第 5、第 3 位」更本质），而且便于外推到比训练时更长的序列（第 23 章的长上下文外推、YaRN 全建立在 RoPE 上）。config 里的 `rope_theta`（RoPE 的底数 θ ）就是它的关键超参。

6.2 RMSNorm：LayerNorm 砍掉一半

一句话回顾（详见第 8 章第 4 节）：RMSNorm 相对 LayerNorm 去掉了「减均值」和 bias β ，只用均方根（root mean square， $\text{RMS}(x) = \sqrt{\frac{1}{d} \sum_i x_i^2}$ ）做缩放：

$$\text{RMSNorm}(x) = \gamma \odot \frac{x}{\sqrt{\frac{1}{d} \sum_i x_i^2 + \epsilon}}$$

其中 γ 是可学习的缩放向量（长度 d ）， ϵ 是防止除零的小常数——记号与第 8 章第 4.4 节一致。

换在哪：原版和 GPT 系列用 LayerNorm，LLaMA / Qwen 换成 RMSNorm。动机很实在——更省、更快，效果基本不掉：少算一个均值、少一组 bias 参数，在几十上百层里累积起来就是可观的计算节省。config 里的 `rms_norm_eps` 就是上面公式里的 ϵ 。位置上它仍然是 Pre-Norm（GPT-2 定下的 Pre-LN 传统，只是把 Norm 的型号从 LayerNorm 换成 RMSNorm）。

6.3 SwiGLU：给 FFN 加一道门控

一句话回顾（详见第 8 章第 5 节）：SwiGLU 把 FFN 的「升维 → 激活 → 降维」升级成门控结构，用一条 SiLU 激活的「门」去逐元素调制另一条线性变换：

$$\text{FFN}_{\text{SwiGLU}}(x) = (\text{SiLU}(xW_{\text{gate}}) \odot (xW_{\text{up}})) W_{\text{down}}$$

因为多了一个门控矩阵 W_{gate} ，为对齐参数量，中间维度 d_{ff} 从原版的 $4d_{\text{model}}$ 收到约 $\frac{8}{3}d_{\text{model}}$ （这是 LLaMA 的惯例值，不同模型略有出入，如 Qwen3-8B 约 $3d_{\text{model}}$ ）。

换在哪：激活函数这条线是 ReLU（原版）→ GELU（GPT 系列）→ SiLU（LLaMA / Qwen）；而从 SiLU 这一代起还额外套了一层门控，激活加门控合起来才叫 SwiGLU——所以严格说，换掉的不只是激活函数，是整个 FFN 的形式。动机是同样参数量下效果更好，代价是 FFN 里多一个矩阵（三个投影而非两个）。config 里 `hidden_act` 为 `silu`、且有门控 / 升维 / 降维三个投影 $W_{\text{gate}} / W_{\text{up}} / W_{\text{down}}$ （代码里叫 `gate_proj` / `up_proj` / `down_proj`），就是 SwiGLU 的标志。

6.4 GQA：为省 KV cache 削减 K/V 头

一句话回顾（详见第 7 章第 5 节）：GQA（分组查询注意力）让 query 头数保持 H 不变、只削减 K/V 头数到 G 组，每组 K/V 被多个 query 头共享（ $G = H$ 即退化回 MHA、 $G = 1$ 即 MQA）。

换在哪：原版和 GPT 系列用标准 MHA（Q/K/V 头数相等）。LLaMA-2 的 34B / 70B 两个型号、以及 Qwen2 起各型号换成 GQA（LLaMA-1 和 LLaMA-2 的 7B / 13B 还是 MHA——GQA 是从「模型大到 KV cache 占用成为瓶颈」才开始划算的）。这里有一点要说明：34B 这个型号只存在于 LLaMA-2 的论文里——论文给了它的配置和评测成绩，但 Meta 最终没有发布它的权重（官方理由是安全评估没做完），所以实际能下载到的 GQA 版 LLaMA-2 只有 70B。本章几处提到 34B，指的都是论文里那份配置。GQA 的动机是推理时 KV cache 太占显存——KV cache 的大小正比于 K/V 头数，把 K/V 头砍到 1/4，KV cache 就省 3/4，长上下文推理才吃得消（第 14 章）。config 里 `num_attention_heads`（query 头数）大于 `num_key_value_heads`（KV 头数）就是 GQA 的标志；Qwen3-8B 是 32 query 头 / 8 KV 头。

6.5 MoE：把一个 FFN 换成一堆稀疏专家

一句话回顾（详见第 9 章第 2.5 节，第 22 章展开）：MoE（混合专家）把每层那一个 FFN 换成 E 个并列的「专家 FFN」加一个路由器，每个 token 只被路由到其中 k 个专家去算。这样总参数量能涨十几倍、但单个 token 的计算量几乎不变——「模型很大，但每次只调用一小部分」。

换在哪：这是可选项，不是所有型号都用。原版 / GPT-1/2/3 / LLaMA 稠密版 / Qwen 稠密版都是每层一个稠密 FFN；而 Qwen3 的 MoE 版、DeepSeek-V3 等把 FFN 换成 MoE。它动的只是 FFN 子层，注意力、残差、归一化那套骨架原封不动——这也再次说明这套 decoder-only 骨架有多通用。细节留到第 22 章展开。

6.6 一堆小改动：去 bias、QK-norm、weight tying

除了上面五个「大件」，现代模型还有一批不改变整体形状的小改良（第 9 章第 2.5 节列过，这里归拢一下）：

- 去掉线性层的 bias：LLaMA 与 Qwen3 的线性投影（Q/K/V/O、FFN 的三个投影）都不带 bias。少一组 bias 参数、对效果几乎无影响，还略微稳一点。顺带一提，Qwen 早期版本（Qwen1 / Qwen2）的 Q/K/V 投影是带 bias 的，到 Qwen3 才去掉、换成下面的 QK-norm——可见「去 bias」也不是一步到位的。
- QK-norm：在 Q、K 投影之后各加一道 RMSNorm 再去算注意力（Qwen3 用了），让注意力分数的尺度更稳、长训练更不容易发散。
- weight tying（权重共享）：token embedding 和输出头 `lm_head` 共享同一个矩阵（互为转置，第 4 章第 1.3 节讲过）。小模型上省参数明显（两端各占词表 $\times d_{\text{model}}$ ），所以 Qwen3 里参数量小的几个型号（0.6B / 1.7B / 4B）默认共享、参数量大的（8B 及以上）默认不共享——config 里 `tie_word_embeddings` 字段记录这个选择。

这些小改动单看每个都不起眼，但「能省就省、能稳就稳」的工程哲学，正是现代大模型配方的底色。

6.7 这些零件是谁提出的：出处与时间差

上面每一项我们都说了「LLaMA / Qwen 换成了 X」，但有一点必须讲清楚，否则容易造成误解：这些零件基本都不是 LLaMA 或 Qwen 发明的。它们大多出自更早的专门论文，LLaMA / Qwen 做的事是把这些零散的改良挑出来、组装成一套完整配方并做到最大规模：

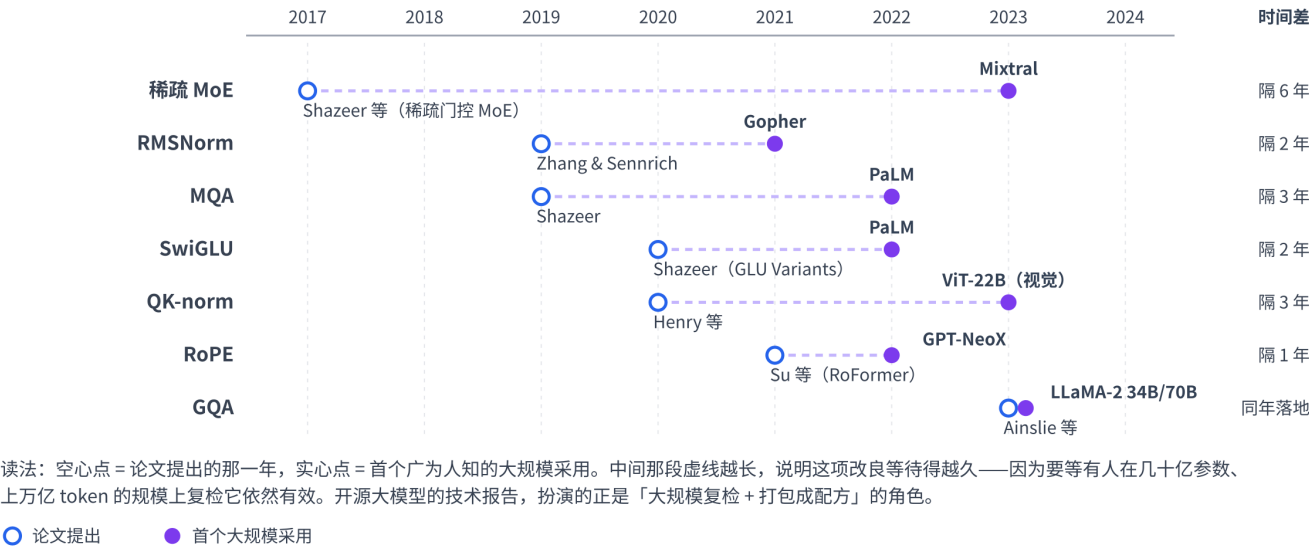
零件	提出它的论文	提出年份	首个广为人知的大规模采用
decoder-only 语言模型	《Generating Wikipedia by Summarizing Long Sequences》（Liu 等）	2018.01	GPT-1（2018.06）
GELU	《Gaussian Error Linear Units (GELUs)》（Hendrycks & Gimpel）	2016	GPT-1 / BERT（2018）
Pre-LN	无单一出处：GPT-2（2019）把它带成主流默认，《On Layer Normalization in the Transformer Architecture》（Xiong 等，2020）补上理论分析	2019–2020	GPT-2（2019）
RMSNorm	《Root Mean Square Layer Normalization》（Zhang & Sennrich）	2019	Gopher（2021）、LLaMA（2023）让它普及
MQA	《Fast Transformer Decoding: One Write-Head is All You Need》（Shazeer）	2019	PaLM（2022）

零件	提出它的论文	提出年份	首个广为人知的大规模采用
SwiGLU	《GLU Variants Improve Transformer》 (Shazeer)	2020	PaLM (2022) 、LLaMA (2023)
RoPE	《RoFormer: Enhanced Transformer with Rotary Position Embedding》 (Su 等)	2021	GPT-NeoX (2022) 、PaLM (2022) 、LLaMA (2023)
GQA	《GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints》 (Ainslie 等)	2023	LLaMA-2 34B / 70B (2023)
QK-norm	《Query-Key Normalization for Transformers》 (Henry 等) 提出雏形 (对 Q、K 做 L2 归一化) ；今天通行的「Q、K 上各加一道 LayerNorm / RMSNorm」由 ViT-22B (2023) 确立	2020	ViT-22B (2023, 视觉)、OLMo 2 (2024)、Qwen3 (2025)
稀疏 MoE	《Outrageously Large Neural Networks》 (Shazeer 等, 2017) 提出稀疏门控, 《Switch Transformers》 (Fedus 等, 2021) 给出 Transformer 上的现代形态	2017–2021	Mixtral (2023)、DeepSeek-V3 (2024)、Qwen3-MoE (2025)

这张表里最值得琢磨的是那个时间差——把它画到时间轴上看得更清楚（空心点是论文提出、实心点是首个大规模采用，中间那段虚线就是从提出到被采用的等待时间）：

现代零件的出处与「时间差」

这些零件都不是 LLaMA / Qwen 发明的——大多出自更早的专门论文；从「有论文」到「被大规模用上」，中间往往还要再等好几年



慢的可以很慢：稀疏 MoE 2017 年就有论文，到 2023 年 Mixtral 才让它广为人知，隔了 6 年；QK-norm 2020 年提出，2023 年先在视觉模型 ViT-22B 上流行起来，2024 年才进入开源 LLM（OLMo 2）。快的当然也有：RoPE 2021 年提出、2022 年 GPT-NeoX 就用上了，GQA 更是 2023 年提出、同年就进了 LLaMA-2。但总体看，一项改良从「有论文」到「被大规模用上」，隔个两三年是常态。

为什么会有这个时间差？原因不难理解：单篇论文通常只在小模型上验证了「有一点提升」，而真正决定它能不能进主流配方的，是有没有人在几十亿参数、上万亿 token 的规模上验证它依然有效——这种验证的成本，只有做大模型的团队付得起。LLaMA / Qwen 这类开源大模型的技术报告，扮演的正是这个「大规模复检 + 打包成配方」的角色。所以读一个新模型的 technical report，最有价值的往往不是它发明了什么，而是它从过去几年的论文里挑中了哪几项、又放弃了哪几项。

6.8 零件对照矩阵：原版 / GPT-2 / LLaMA / Qwen3

把本章两条线的结论汇成一张矩阵——每一列是一个模型，每一行是一个零件，一眼看清「哪一项是哪一代换上的」：

零件	原版 Transformer (2017)	GPT-2 (2019)	LLaMA (2023)	Qwen3 (2025)
整体架构	encoder-decoder 两栈	decoder-only	decoder-only	decoder-only
归一化位置	Post-LN	Pre-LN	Pre-LN	Pre-LN
归一化类型	LayerNorm	LayerNorm	RMSNorm	RMSNorm
FFN 形式	ReLU 激活	GELU 激活	SwiGLU (SiLU + 门控)	SwiGLU

零件	原版 Transformer (2017)	GPT-2 (2019)	LLaMA (2023)	Qwen3 (2025)
位置编码	sinusoidal 绝对	learned 绝对	RoPE	RoPE
注意力	MHA	MHA	MHA → GQA (LLaMA-2 的 34B / 70B)	GQA
线性层 bias	有	有	无	无
QK-norm	无	无	无	有
FFN 变体	稠密	稠密	稠密	稠密 / MoE (可选)

加粗的单元格标出「这一项在这一代首次换上」。读这张表要先明确两件事：

- 「首次」是相对这张表的四列而言的，指「在原版 → GPT-2 → LLaMA → Qwen3 这条采样出来的路径上，它是从这一代开始变的」，不等于这个零件是这一代发明的——每项零件真正的出处论文与年份见上面第 6.7 节那张表（比如 RoPE 加粗在 LLaMA 列，但它是 2021 年 RoFormer 提出的）。
- decoder-only、GELU、learned 绝对位置其实都是 GPT-1（2018，表中未单列）首先用上的，GPT-2 只是沿用，所以这三项在 GPT-2 列都不加粗（表里能看到它们相对原版确实变了，但「首次」不落在 GPT-2）。

这个表格就是一部浓缩的大模型架构演进史：GPT-2 立起 Pre-LN，LLaMA 一口气换上 RMSNorm / SwiGLU / RoPE / GQA / 去 bias 这套现代零件，Qwen3 再补上 QK-norm 和可选的 MoE。骨架自 2017 年就没变过，变的全是零件的型号。

七、实战：从 config 读懂演进

本章实战不训练、不推理，只做一件事：用 `AutoConfig` 把上面几张表里的「论文说法」和真实模型 `config.json` 里的「字段值」一一对上。全程 CPU、只下载几 KB 的 json。

7.1 环境自检与依赖

Cell 0 是常规环境自检——本章纯 CPU，只打印环境信息、不强制 GPU：

```
# =====
# Cell 0: 环境自检 (本章纯 CPU 即可, 无需 GPU)
# =====
# 本章只读几个模型的 config.json (各几 KB) 并画一张小图, 全程 CPU 秒级完成,
# 不下载任何模型权重、不做任何前向 / 反向。
import sys, platform
import torch
```

```
print("Python:", sys.version.split()[0])
print("平台:", platform.platform())
print("PyTorch:", torch.__version__)
print("CUDA 可用:", torch.cuda.is_available(), " (本章用不到, CPU 即可) ")
```

Cell 1 装依赖。本章只用 `transformers` 的 `AutoConfig` 读 config, 外加 `matplotlib` 画一张图:

```
%%capture
# =====
# Cell 1: 安装依赖
# =====
# %%capture 必须是 cell 第一行, 把 pip 安装日志藏起来。
# transformers: 用 AutoConfig 读 config; GPT-2 是老模型不挑版本, Qwen3 要求 >=4.51,
# 故统一锁 >=4.51 一次装好。matplotlib / torch: Colab 默认已装。
!pip install -q -U "transformers>=4.51"
```

7.2 读 GPT-2 config: 早期 decoder-only 长什么样

Cell 2 先读 GPT-2 (HuggingFace 上的 `gpt2`, 即 124M 的最小版本), 看看 GPT 系列早期这套「decoder-only + learned 绝对位置 + GELU」的配置。注意 GPT-2 的 config 字段名和现代模型不一样 (`n_layer` / `n_embd` / `n_head`)。

```
# =====
# Cell 2: 读 GPT-2 config, 看早期 decoder-only 的配置
# =====
from transformers import AutoConfig

# rope_theta (RoPE 的 base  $\theta$ ) 在 config 里的位置随 transformers 版本而变: 4.x 放在顶层
# cfg.rope_theta; 5.x 起挪进嵌套字典 cfg.rope_parameters["rope_theta"]。下面这个小工具两处都试,
# 兼容新旧版本; 两处都没有就返回 None——说明这个模型根本不用 RoPE。Cell 3 读 Qwen3 时还会再用一次。
def get_rope_theta(cfg):
    theta = getattr(cfg, "rope_theta", None)
    if theta is None:
        theta = (getattr(cfg, "rope_parameters", None) or {}).get("rope_theta")
    return theta

# gpt2 = GPT-2 最小的那个版本 (124M)。AutoConfig 只下载 config.json (几 KB), 不下载权重。
gpt2 = AutoConfig.from_pretrained("gpt2")

print("GPT-2 (124M) 关键配置: ")
print(f"  层数      n_layer      = {gpt2.n_layer}")
print(f"  隐藏维度   n_embd       = {gpt2.n_embd}")
print(f"  注意力头数 n_head       = {gpt2.n_head}")
print(f"  上下文长度 n_positions  = {gpt2.n_positions}")
print(f"  词表大小   vocab_size    = {gpt2.vocab_size}")
print(f"  激活函数   activation_function= {gpt2.activation_function}") # gelu_new
# GPT-2 的位置编码是 learned 绝对: config 里体现为有一张 n_positions 长的位置 embedding 表
# (wpe), 而不像现代模型那样带 rope_theta。下面这句确认它两处【都】没有 rope 参数。
print(f"  RoPE base   rope_theta    = {get_rope_theta(gpt2)} (None => 用 learned 绝对位置, 不是 RoPE)")
print(f"  归一化 eps  layer_norm_epsilon = {gpt2.layer_norm_epsilon} (LayerNorm, 不是 RMSNorm)")
```

跑出来能看到：这个 124M 版本是 12 层、 $d_{\text{model}} = 768$ 、12 个注意力头、上下文 1024、GELU 激活、LayerNorm、`rope_theta` 取出来是 **None**（用的是 learned 绝对位置）。注意这里读的是 124M 的最小版本，层数 / 维度都是它自己的数，和第 4.3 节那张表对不上——那张表里 GPT-2 那一列取的是 1.5B 的最大版本（gpt2-xl）：48 层、 $d_{\text{model}} = 1600$ 、25 个注意力头。两者是同一个 GPT-2 家族里的不同型号，同一套结构、只是宽度和深度不同。真正能和表对上的是定性特征——位置编码还是绝对的、激活还是 GELU、归一化还是 LayerNorm，现代那套零件一个都还没换上。

7.3 读 Qwen3-8B config：现代改动逐项对上

Cell 3 再读一次 Qwen3-8B（第 11 章末尾读过一次，这里换个角度：逐项对上第 6 节的「现代改动」清单）。

```
# =====
# Cell 3: 读 Qwen3-8B config, 逐项对上第 6 节的现代改动
# =====
from transformers import AutoConfig

cfg = AutoConfig.from_pretrained("Qwen/Qwen3-8B")

# 复用 Cell 2 里定义的 get_rope_theta(): 顶层 cfg.rope_theta 与嵌套 cfg.rope_parameters["rope_theta"]
# 两处都试, 所以不管装的是 transformers 4.x 还是 5.x 都能取到值。
rope_theta = get_rope_theta(cfg)

print("Qwen3-8B 关键配置 (对上第 6 节现代改动清单): ")
print(f"  层数          num_hidden_layers  = {cfg.num_hidden_layers}")
print(f"  隐藏维度       hidden_size       = {cfg.hidden_size}")
print(f"  FFN 中间维度   intermediate_size = {cfg.intermediate_size}")
print(f"  query 头数     num_attention_heads = {cfg.num_attention_heads}")
print(f"  KV 头数       num_key_value_heads = {cfg.num_key_value_heads}")
print(f"  RoPE base     rope_theta         = {rope_theta}")
print(f"  归一化 eps     rms_norm_eps       = {cfg.rms_norm_eps}")
print(f"  激活函数       hidden_act         = {cfg.hidden_act}")
print(f"  词表大小       vocab_size          = {cfg.vocab_size}")
print(f"  权重共享       tie_word_embeddings = {cfg.tie_word_embeddings}")

print("\n逐项对上第 6 节:")
print(f"  6.1 RoPE      : 有 rope_theta={rope_theta}      => 用 RoPE (非绝对位置)")
print(f"  6.2 RMSNorm   : 有 rms_norm_eps={cfg.rms_norm_eps} => 用 RMSNorm (非 LayerNorm)")
print(f"  6.3 SwiGLU    : hidden_act={cfg.hidden_act}      => SiLU 门控, 即 SwiGLU (非 ReLU/GELU)")
print(f"  6.4 GQA       : {cfg.num_attention_heads} query 头 > {cfg.num_key_value_heads} KV 头 => GQA (非 MHA)")
print(f"  6.5 MoE       : Qwen3-8B 是稠密版、无 MoE 字段, 故此处不列 (MoE 版才有 num_experts 等字段)")
print(f"  6.6 tying     : tie_word_embeddings={cfg.tie_word_embeddings} => 8B 参数量不小, 默认不共享")
```

对比 Cell 2 和 Cell 3 两段输出，第 6 节讲的每一项改动都能在 config 里找到对应字段：Qwen3-8B 有 `rope_theta` (RoPE)、有 `rms_norm_eps` (RMSNorm)、`hidden_act` 是 `silu` (SwiGLU)、`num_attention_heads` (32) > `num_key_value_heads` (8) (GQA) ——而 GPT-2 这些字段要么没有、要么是老式的 LayerNorm / GELU / 绝对位置。

7.4 画 GPT-1/2/3 参数量增长

Cell 4 把第 4.3 节那张表里的参数量画成图，直观感受「三代涨了约 1500 倍」是什么概念。参数量跨度太大，用对数纵轴才看得清。

```
# =====
# Cell 4: 可视化 GPT-1/2/3 的参数量增长（对数纵轴）
# =====

import matplotlib.pyplot as plt

models = ["GPT-1\n(2018)", "GPT-2\n(2019)", "GPT-3\n(2020)"]
params_b = [0.117, 1.5, 175.0]  # 单位：十亿 (B) 参数：117M / 1.5B / 175B

fig, ax = plt.subplots(figsize=(7, 4.5))
bars = ax.bar(models, params_b, color=["#93c5fd", "#60a5fa", "#2563eb"])
ax.set_yscale("log")  # 参数量跨 3 个数量级，线性轴会把前两根压成 0，必须用对数轴
ax.set_ylabel("Parameters (Billions, log scale)")
ax.set_title("GPT-1 -> GPT-2 -> GPT-3: ~1500x parameter growth in 2 years")
# 在每根柱子顶上标注具体数值
for b, p in zip(bars, params_b):
    label = f"{p*1000:.0f}M" if p < 1 else f"{p:.1f}B"
    ax.text(b.get_x() + b.get_width() / 2, p * 1.15, label, ha="center", va="bottom")
plt.tight_layout()
plt.show()

# 观察：纵轴每一格是 10 倍。GPT-1(117M) -> GPT-2(1.5B) 约 13 倍，
# GPT-2 -> GPT-3(175B) 约 117 倍，两年累计约 1500 倍——而架构几乎没变。
# 这张图就是「规模是 GPT 系列主旋律」最直接的证据。
```

上面这几段代码合起来就是本章两条线的实测版：GPT 系列（第 2-5 节）把 decoder-only 骨架立了起来，LLaMA / Qwen（第 6 节）把骨架里的零件逐个换新。你现在拿到任何一个新模型的 config，都能照着第 6.8 节那张矩阵快速判断它「用的是哪一代的哪套零件」。

八、关键概念回顾

- 系列论文的读法：盯代际之间的 delta——每一代只问三件事「上一代瓶颈是什么、这一代改了哪几件事、多出了什么新能力」，不必逐字读完每篇。
- GPT-1（2018）：首次证明「decoder-only + 大规模自回归预训练 + 下游微调」这套两阶段范式能大幅超过专用模型。奠定了 decoder-only 骨架。
- 预训练—微调范式：先在海量无标注文本上做自回归预训练（无需人工标注的监督信号），再用少量标注数据微调到下游任务。今天的 SFT / LoRA 是它的延续。
- BERT 这条并行路线：同期（2018）Google 的 BERT 走 encoder-only + 掩码语言建模（MLM），是双向的，在理解类任务上一度压过 GPT。最终是 GPT 的单向路线通向通用大模型（能把任务统一成「续写」、规模放大后涌现 in-context learning）；BERT 那条路今天出现在检索 / embedding 模型里。
- GPT-2（2019）：骨架几乎不变、规模拉到 1.5B，提出「语言模型是无监督多任务学习器」，展示 zero-shot。架构上把 Pre-LN 定为后续模型的默认配置（外加 final norm、 $1/\sqrt{N}$ 残差初始化缩放）。

- GPT-3 (2020)：规模再放大到 175B，涌现出 in-context learning——prompt 里给几个示范 (few-shot)，不更新参数就能做新任务。「量变引起质变」的经典案例。
- in-context learning (上下文学习)：把示范写进 prompt、模型在一次前向里现学现用，全程不更新参数。分 zero-shot / one-shot / few-shot 三档；随规模增大才逐渐明显。是 prompt 工程、few-shot、CoT 的基础。
- GPT 这条线的关键词是「骨架 + 规模」：三代参数量涨约 1500 倍，架构实质改动只有 GPT-2 那处 Post-LN → Pre-LN，位置编码三代都还是 learned 绝对。
- 从 GPT-3 到 ChatGPT 那一步不在架构上：靠的是 InstructGPT (2022) 那条后训练路线 (SFT + 奖励模型 + RLHF)，骨架一个零件没动，留到第 27-31 章 (后训练与对齐这一整个阶段是第 27-36 章)。GPT-4 起闭源、不公开架构，所以架构演进这条线只能靠开源模型继续读下去。
- LLaMA / Qwen (2023 起) 的现代零件：RoPE (绝对 → 旋转相对位置，第 4 章第 6 节)、RMSNorm (LayerNorm 砍掉减均值和 bias，第 8 章第 4 节)、SwiGLU (给 FFN 加 SiLU 门控，第 8 章第 5 节)、GQA (削减 K/V 头省 KV cache，第 7 章第 5 节)、可选 MoE (稠密 FFN → 稀疏专家，第 9 章第 2.5 节、第 22 章)，外加去 bias / QK-norm / weight tying 等小改动。
- 零件的出处与「时间差」：这些零件都不是 LLaMA / Qwen 发明的——RMSNorm (2019)、MQA (2019)、SwiGLU (2020)、RoPE (2021)、GQA (2023) 各有更早的专门论文。一项改良从「有论文」到「成为标配」通常隔两三年，因为要等有人在大规模上复检它依然有效。开源大模型的技术报告扮演的正是「大规模复检 + 打包成配方」的角色。
- 零件对照矩阵：原版 / GPT-2 / LLaMA / Qwen3 逐行对照，加粗标出每项零件「首次换上」的那一代——注意「首次」是相对这四列而言，不等于该零件由这一代发明。
- config 字段 ↔ 论文改动的对应：`rope_theta` ↔ RoPE、`rms_norm_eps` ↔ RMSNorm、`hidden_act=silu` ↔ SwiGLU、`num_attention_heads > num_key_value_heads` ↔ GQA、`tie_word_embeddings` ↔ weight tying。读 config 就能反推一个模型用了哪套零件。

九、本章小结

- 本章不引入新组件，做的是一次「论文串读」：顺两条时间线，把大模型从 2017 原版走到今天 Qwen3 的演进串成一条完整叙事。凡是前面介绍过的零件都只做一个简单回顾，重点是那条贯穿多篇论文的脉络。
- 第一条线 GPT-1 → GPT-2 → GPT-3，讲 decoder-only 路线怎么确立：GPT-1 证明「decoder-only + 预训练」能走通，GPT-2 证明「规模够大能不微调直接做任务 (zero-shot)」，GPT-3 证明「规模再大两个数量级会涌现 in-context learning (few-shot)」。三代把 decoder-only 从「一种选择」推成了「事实标准」，而这一路上架构基本冻结、规模是主旋律 (参数量涨约 1500 倍，实质架构改动只有 Pre-LN 一处)。
- 第一条线到 GPT-3 为止是有原因的：GPT-3 → ChatGPT 那一步改的是训练流程 (SFT + RLHF，第 27-31 章) 而非架构，GPT-4 之后又不再公开架构细节。所以架构演进这条线自然接到了开源的 LLaMA / Qwen 上。
- 第二条线原版 → LLaMA / Qwen，讲骨架定型后怎么换零件：decoder-only 赢下主赛道后，改进从「选架构」转向「在固定骨架里换更好的零件」——RoPE、RMSNorm、SwiGLU、GQA、可选 MoE，外加去

bias / QK-norm / weight tying。这些零件前面各章都介绍过，本章把它们凑成一张对照矩阵，看清每一项是哪一代首次换上的；同时也点明它们大多出自比 LLaMA 更早的专门论文（GQA 是同年落地的例外），LLaMA / Qwen 的贡献是「在最大规模上复检并打包成一套配方」。

- 一张零件对照矩阵收束全章：原版（2017） / GPT-2（2019） / LLaMA（2023） / Qwen3（2025）逐行对照，加粗标出「首次换上」——decoder-only、GELU、learned 绝对位置都始于 GPT-1（表中未单列），GPT-2 首次立起 Pre-LN，LLaMA 一口气换上 RMSNorm / SwiGLU / RoPE / GQA / 去 bias，Qwen3 补上 QK-norm 与可选 MoE。骨架自 2017 未变，变的全是零件型号。
- 实战我们用 **AutoConfig** 读了 GPT-2 和 Qwen3-8B 的 config，把「论文说换了什么」和「config 里改了哪个字段」一一对上，并画了 GPT-1/2/3 参数量增长图。掌握这套「读 config 反推零件」的本事，你以后拿到任何新模型都能快速判断它的架构谱系。

到这里，第 3-12 章这段「Transformer 架构原理」就讲完了：我们从 tokenizer 一路讲到整体架构（第 3-9 章）、讲清了自回归训练目标（第 10 章）、精读了奠基论文（第 11 章）、串读了 GPT 与现代模型的演进（本章）。零件、总装、目标、源流，四件事齐了。下一章第 13 章是这一阶段的收官之作，咱们把这些知识落到最实处——从零实现一个 mini-GPT，在 tiny-shakespeare 数据集上完整跑通「训练 → 生成」的闭环，亲手验证这一整段学到的所有东西。